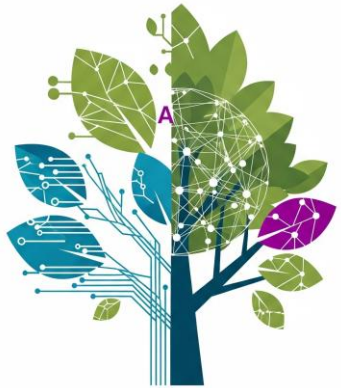
GEOMATYKA I SZTUCZNA INTELIGENCJA W LEŚNICTWIE

Geomatics and Artificial Intelligence in Forestry



Programowanie w R

Bartłomiej Kraszewski

Zakład Geomatyki, Instytut Badawczy Leśnictwa

Plan

09:00 – 11:00	Fundamenty i typy danych (2h)
11:30 – 13:00	Manipulacja danymi (1.5 h)
14:00 – 15:30	Wizualizacja Podstawy programowania (1.5 h)
16:00 – 17:00	Wprowadzenie do GIS w R Zadanie projektowe (1 h)

FUNDAMENTY

Czym właściwie jest R?

R to język programowania i środowisko

do obliczeń statystycznych oraz wizualizacji danych.



Czym właściwie jest R?

R to środowisko **Open Source**

Powstał w 1993 roku w Auckland (Nowa Zelandia).

Nazwa pochodzi od imion twórców:
Rossa Ihaki i **Roberta Gentlemana**.

Stworzony przez statystyków, dla statystyków.

Dlaczego R zyskał taką popularność?

PYPL PopularitY

of Programming Language

Worldwide, Apr 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	36.21 %	+5.7 %
2	↑↑	C/C++	13.21 %	+6.2 %
3	↓	Java	10.01 %	-5.4 %
4	↑↑	R	6.17 %	+1.6 %
5	↓↓	JavaScript	5.07 %	-3.0 %
6	↑↑↑↑↑	Swift	3.15 %	+0.8 %
7	↓↓	C#	3.0 %	-3.0 %
8		Rust	2.98 %	-0.1 %
9	↓↓	PHP	2.96 %	-0.7 %
10		Objective-C	2.57 %	+0.1 %

Dlaczego R zyskał taką popularność?

Obsługuje **zbiory danych**, z którymi arkusze kalkulacyjne sobie nie radzą

Raz napisany skrypt przetworzy nowe dane **dokładnie w ten sam sposób**

Możliwość tworzenia **wykresów** o jakości publikacyjnej

Brak opłat licencyjnych, w przeciwieństwie do SAS czy SPSS

Jak to działa?

CRAN

The Comprehensive R Archive Network

Ogromne globalna sieć serwerów przechowująca

ponad **20 000 darmowych pakietów**

https://cran.r-project.org/

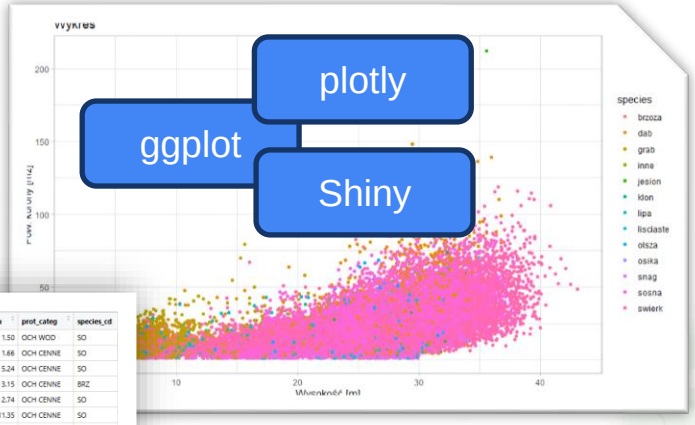
Jak to działa?

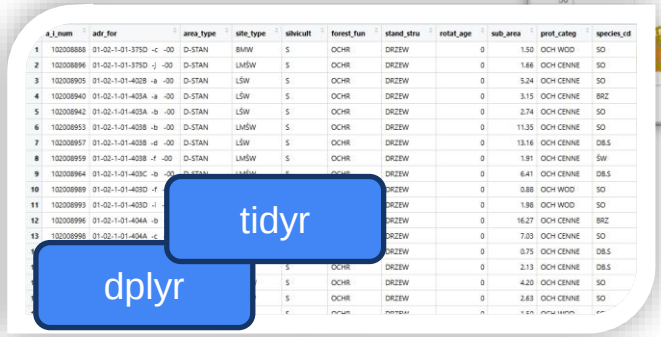
Rank	Package Name	Downloads	Author	Maintainer
1	ggplot2	184,171,804		
2	rlang	173,300,886		
3	magrittr	153,342,699		
4	dplyr	143,525,076		
5	cli	130,168,285		
6	vctrs	130,028,726		
7	tibble	123,972,400		
8	lifecycle	119,289,684		
9	jsonlite	118,484,591		
10	Rcpp	116,375,089		

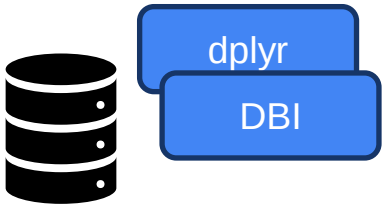
https://www.r-pkg.org/downloaded

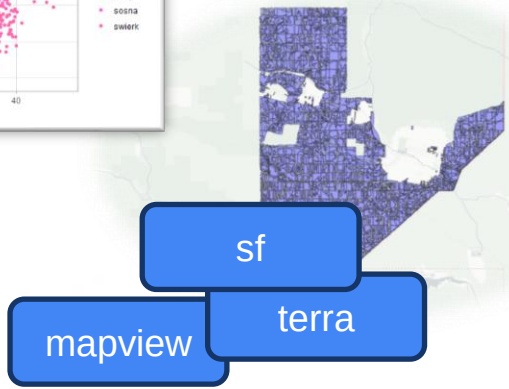
Jak to działa?

Pakiety to aplikacje wewnątrz języka.

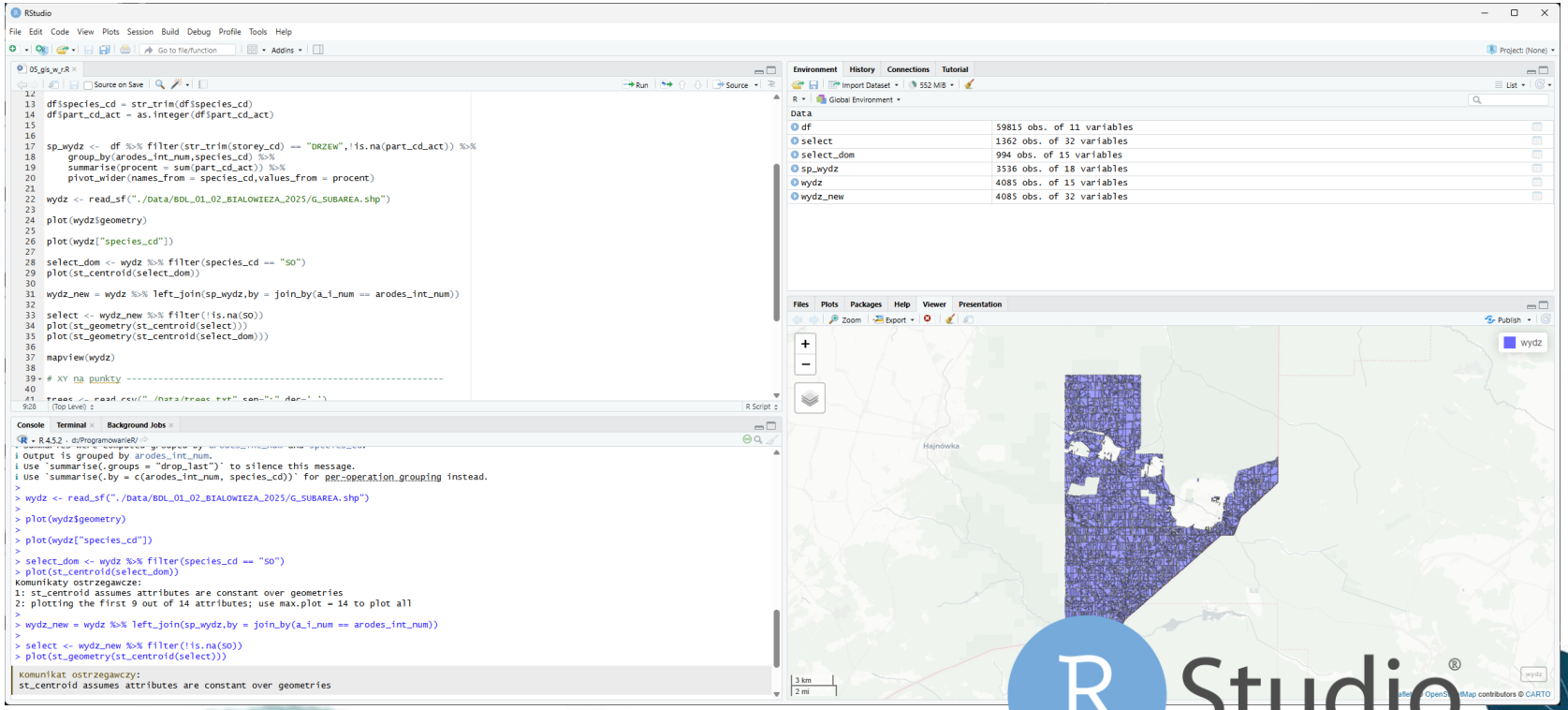




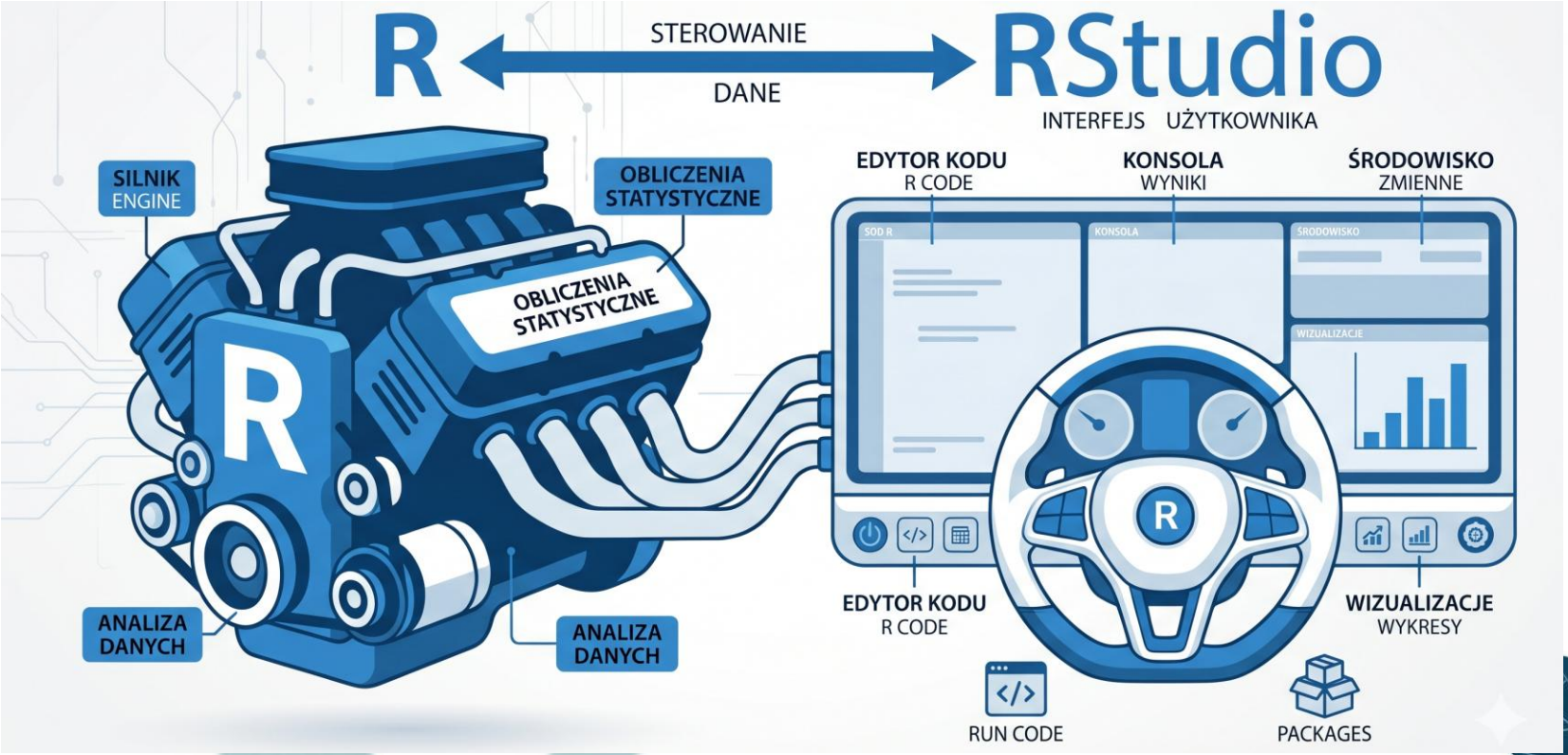




Jak to działa?



Jak to działa?



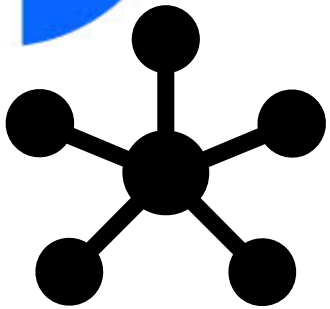
Dla kogo?

- Data Scientists
- Statystycy
- Bioinformatycy
- Analitycy finansowi
- Przyrodnicy
- Naukowcy

Kto używa?

Google

The New York Times



NAUKA

Przypisanie <-

W większości języków (Python, JS, C#) używamy znaku =.

W R:

<- to główny operator przypisania wartości do obiektu.

= jest używany niemal wyłącznie do definiowania argumentów wewnątrz funkcji (np. `mean(x = c(1, 2, 3))`).



Przypisanie <-

Strzałka wskazuje, dokąd "płyną" dane

Używanie = do przypisania zmiennych jest technicznie możliwe, ale uznawane za błąd w stylu kodowania R

W R <- jest standardem. Pozwala wyraźnie odróżnić przypisanie do zmiennej od definiowania argumentów wewnątrz funkcji

Alt + minus (-)

Indeksowanie zaczynamy od 1

To najważniejsza różnica dla osób przyzwyczajonych do indeksowania od zera (0-based indexing):

- W R pierwszy element wektora lub kolumny ma indeks `[1]`.
- W Pythonie czy C++ pierwszy element to `[0]`.

Dlaczego tak jest?

R został stworzony **przez statystyków dla statystyków**. W matematyce i statystyce naturalne jest, że "pierwszy element to pierwszy", a nie "zerowy".

Komentarze

Kod mówi, CO robisz. Komentarze mówią, DLACZEGO to robisz.

W R do komentarzy używamy symbolu **hash (#)**. Wszystko, co znajduje się po prawej stronie tego znaku w danej linii, jest ignorowane przez program.

Ctrl + Shift + C

TYPY DANYCH

Wstęp do typów danych

R jest językiem

dynamicznie typowanym

Typy liczbowe

Numeric (Double): Liczby rzeczywiste

```
x <- 5.5
```

Integer: Liczby całkowite

```
x <- 5L
```

Wartości specjalne:

```
Inf, NaN
```

Typ tekstowy (Character)

Przechowuje **ciągi znaków**. To jeden z najczęściej używanych typów w analizie danych tekstowych.

Używamy cudzysłowów (" ") lub apostrofów (' ,).

```
gatunek <- „sosna"
```

Liczba zapisana w cudzysłowie (np. "10") jest traktowana jako tekst, a nie wartość matematyczna.

Brak informacji

R posiada unikalne podejście do "braku" informacji.

NA (Not Available): Oznacza brakującą wartość w zbiorze (np. brak pomiaru). Zachowuje typ danych (istnieje NA_integer_, NA_character_ itd.).

NULL: Oznacza, że obiekt jest pusty lub nie istnieje. Często używane do "resetowania" list lub kolumn.

Wektor



Wektor to podstawowy „pojemnik” na dane tego samego typu.

Zasada homogeniczności: Wszystkie elementy wektora **muszą** być tego samego typu (np. same liczby albo same teksty).

Wektorowanie: Operacje w R są zoptymalizowane pod kątem wektorów – możesz dodać liczbę do całego wektora jedną komendą

Wektor




```
moje_liczby <- c(10, 20, 30.5, 40)
miasta <- c("Warszawa", "Kraków", "Gdańsk")
wyniki <- c(TRUE, FALSE, TRUE, TRUE)
```

Wektor



`c()`	Tworzy wektora
`sum()`	Suma wszystkich elementów
`sort()`	Sortuje elementy rosnąco (dodaj `decreasing = TRUE` dla malejącego).
`unique()`	Usuwa duplikaty, zwracając tylko unikalne wartości
`which()`	zwraca indeksy elementów, które spełniają dany warunek.

Macierz

matrix()

Macierz to dwuwymiarowa struktura danych,
która (podobnie jak wektor) musi być **homogeniczna**.

Wymiary: Posiada wiersze (*rows*) i kolumny (*columns*).

Typ danych: Wszystkie komórki muszą zawierać ten sam typ
(np. same liczby lub same znaki logiczne).

Zastosowanie: Głównie obliczenia matematyczne,
statystyczne i algebra liniowa.

Factor

factor()

Reprezentacja danych kategorycznych:
Służy do przechowywania zmiennych
o skończonej liczbie unikalnych wartości
(np. płeć, region, stopień naukowy, gatunek).

Struktura dwuwarstwowa:
Widoczna: Etykiety tekstowe (**Labels/Levels**).
Wewnętrzna: Przechowywane jako liczby całkowite (**Integers**), co optymalizuje pamięć.

Factor

factor()

Zarządzanie poziomami:

- Pozwala na zdefiniowanie wszystkich możliwych wartości, nawet jeśli nie występują w danym zbiorze.
- Umożliwia ustalenie niestandardowej kolejności (np. dni tygodnia zamiast porządku alfabetycznego).

Factor

factor()

```
gatunki_drzew <- c("Dąb", "Sosna", "Brzoza", "Dąb", "Brzoza",
"Sosna", "Sosna")
# Konwersja na factor
drzewa_f <- factor(gatunki_drzew)
print(drzewa_f)
# Wynik: [1] Dąb Sosna Brzoza Dąb Brzoza Sosna Sosna
# Levels: Brzoza Dąb Sosna
```

Factor

factor()

`factor()`	Tworzy nowy faktor z wektora
`levels()`	Wyświetla lub zmienia nazwy poziomów
`Nlevels()`	Zwraca liczbę unikalnych poziomów
`is.factor()`	Sprawdza, czy obiekt jest faktorem
`table()`	Tworzy tabelę liczebności dla poziomów

Lista

list()

Heterogeniczność: W przeciwieństwie do wektorów, lista może przechowywać elementy różnych typów (liczby, tekst, wartości logiczne) jednocześnie.

Rekurencyjność: Lista może zawierać inne listy (tworząc struktury drzewiaste).

Dowolność obiektów: Wewnątrz listy mogą znajdować się wektory, macierze, ramki danych, a nawet funkcje.

Podstawa zaawansowanych obiektów: Większość wyników testów statystycznych i modeli to w rzeczywistości bardzo rozbudowane listy.

Lista

list()

```
moje_dane <- list(
id = 1:5,
miasto = "Warszawa",
wyniki = c(95.5, 88.0),
aktywny = TRUE )
```

Lista – kluczowe funkcje

list()

`list()`	Tworzy nową listę
`str()`	Funkcja do podglądu struktury listy.
`names()`	Wyświetla lub nadaje nazwy elementom listy.
`unlist()`	Spłaszcza listę do wektora
`length()`	Zwraca liczbę głównych elementów w liście.

Ramka danych

data.frame()

Data Frame to serce analizy danych w R. Jest to najczęściej używana struktura, która łączy w sobie cechy macierzy (układ dwuwymiarowy) i listy (różne typy danych).

Tabela dwuwymiarowa: Dane zorganizowane w wiersze (obserwacje) i kolumny (zmienne).

Heterogeniczność kolumn: Każda kolumna może zawierać inny typ danych (np. jedna tekst, druga liczby, trzecia wartości logiczne).

Ramka danych

data.frame()

Homogeniczność wewnątrz kolumny: Wszystkie elementy w obrębie jednej kolumny **muszą** być tego samego typu.

Wymóg równej długości: Każda kolumna musi mieć dokładnie tę samą liczbę wierszy.

Struktura listy: Technicznie jest listą wektorów o tej samej długości.

Ramka danych

data.frame()

```
df <- data.frame(
      id = 1:3,
      imie = c("Anna", "Piotr",
      "Ewa"),
      czy_aktywny = c(TRUE,
      FALSE, TRUE)
)
```

Ramka danych – funkcje

data.frame()

`dim(df)`	zwraca wymiary (liczbę wierszy i kolumn)
`nrow(df) / ncol(df)`	liczba wierszy / kolumn
`names(df)`	nazwy kolumn
`cbind() / rbind()`	dodawanie nowych kolumn lub wierszy

MANIPULACJA DANYMI

Czym jest Tidyverse?

Zbiór powiązanych ze sobą pakietów R zaprojektowanych do Data Science.

Wszystkie pakiety dzielą tę samą strukturę danych i design – tidy data.

install.packages("tidyverse")



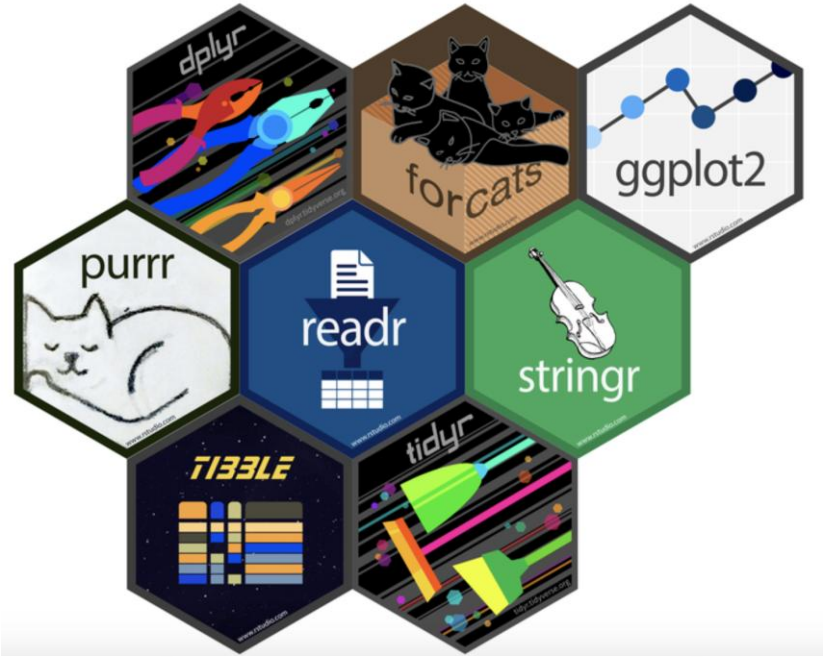
Czym jest Tidyverse?

Uporządkowane zbiory danych są wszystkie do siebie podobne, ale każdy nieuporządkowany zbiór jest nieuporządkowany na swój sposób

Hadley Wickham, główny twórca Tidyverse



Główne pakiety



ggplot2: wizualizacja danych

dplyr: manipulacja danymi

tidyr: porządkowanie struktury danych

readr: import danych

purrr: programowanie funkcyjne

tibble: nowoczesna wersja ramki danych

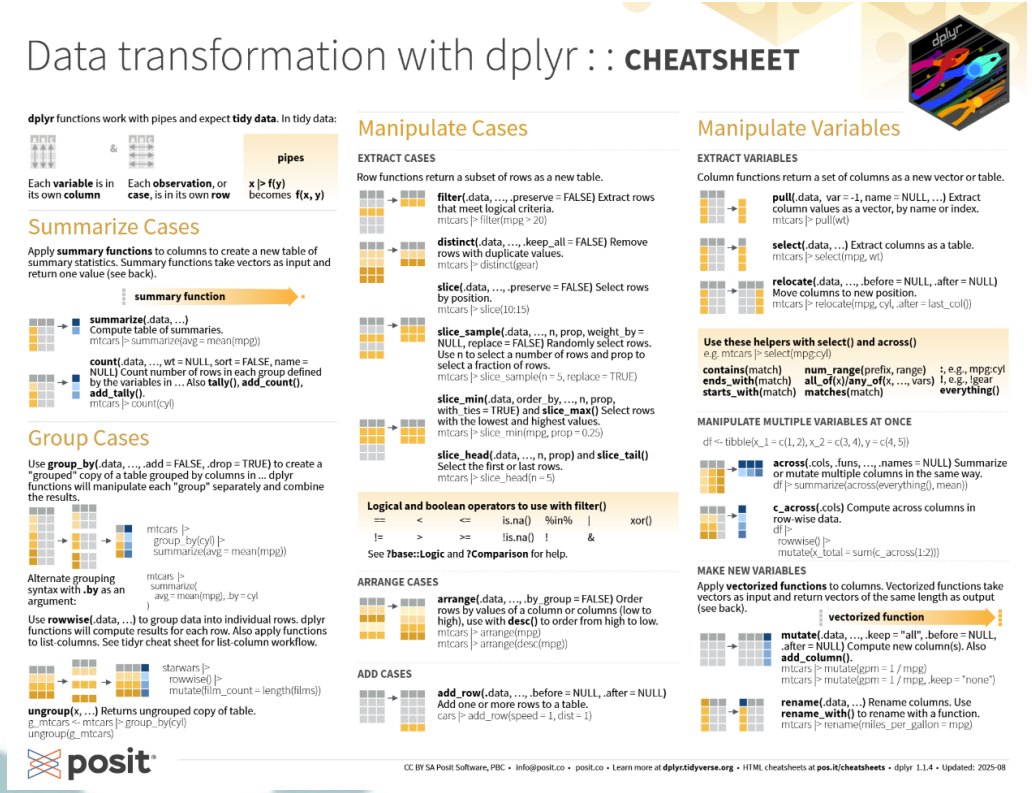
stringr: praca z tekstem

forcats: praca z factorami

Dplyr – Manipulacja danymi

- **`filter()`**: wybieranie wierszy na podstawie warunków
- **`select()`**: wybieranie konkretnych kolumn
- **`mutate()`** : tworzenie nowych kolumn
- **`summarise()`** : agregowanie danych (np. średnia, suma)
- **`arrange()`** : sortowanie wyników
- **`group_by()`** : wykonywanie operacji w grupach

Dplyr cheatsheet



Data transformation with dplyr : : CHEATSHEET

dplyr functions work with pipes and expect **tidy data**. In tidy data:

Each **variable** is in its own **column** & Each **observation**, or **case**, is in its own **row**

pipes
x |> f(y) becomes **f(x, y)**

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function

summarize(.data, …)
Compute table of summaries.
mtcars |> summarize(avg = mean(mpg))

count(.data, …, wt = NULL, sort = FALSE, name = NULL) Count number of rows in each group defined by the variables in … Also **tally()**, **add_count()**, **add_tally()**.
mtcars |> count(cyl)

Group Cases

Use **group_by**(.data, …, .add = FALSE, .drop = TRUE) to create a "grouped" copy of a table grouped by columns in … dplyr functions will manipulate each "group" separately and combine the results.

```
mtcars |>
  group_by(cyl) |>
  summarize(avg = mean(mpg))
```

Alternate grouping syntax with **.by** as an argument:

```
mtcars |>
  summarize(
    avg = mean(mpg), .by = cyl
  )
```

Use **rowwise**(.data, …) to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidyr cheat sheet for list-column workflow.

```
starwars |>
  rowwise() |>
  mutate(film_count = length(films))
```

ungroup(x, …) Returns ungrouped copy of table.

```
g_mtcars <- mtcars |> group_by(cyl)
ungroup(g_mtcars)
```

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.

filter(.data, …, .preserve = FALSE) Extract rows that meet logical criteria.
mtcars |> filter(mpg > 20)

distinct(.data, …, .keep_all = FALSE) Remove rows with duplicate values.
mtcars |> distinct(gear)

slice(.data, …, .preserve = FALSE) Select rows by position.
mtcars |> slice(10:15)

slice_sample(.data, …, n, prop, weight_by = NULL, replace = FALSE) Randomly select rows. Use n to select a number of rows and prop to select a fraction of rows.
mtcars |> slice_sample(n = 5, replace = TRUE)

slice_min(.data, order_by, …, n, prop, with_ties = TRUE) and **slice_max()** Select rows with the lowest and highest values.
mtcars |> slice_min(mpg, prop = 0.25)

slice_head(.data, …, n, prop) and **slice_tail()** Select the first or last rows.
mtcars |> slice_head(n = 5)

Logical and boolean operators to use with filter()

==	<	<=	is.na()	%in%	|	xor()
!=	>	>=	!is.na()	!	&	

See **?base::Logic** and **?Comparison** for help.

ARRANGE CASES

arrange(.data, …, .by_group = FALSE) Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
mtcars |> arrange(mpg)
mtcars |> arrange(desc(mpg))

ADD CASES

add_row(.data, …, .before = NULL, .after = NULL) Add one or more rows to a table.
cars |> add_row(speed = 1, dist = 1)

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.

pull(.data, var = -1, name = NULL, …) Extract column values as a vector, by name or index.
mtcars |> pull(wt)

select(.data, …) Extract columns as a table.
mtcars |> select(mpg, wt)

relocate(.data, …, .before = NULL, .after = NULL) Move columns to new position.
mtcars |> relocate(mpg, cyl, .after = last_col())

Use these helpers with select() and across()
e.g. mtcars |> select(mpg:cyl)

contains(match)
ends_with(match)
starts_with(match)
num_range(prefix, range)
all_of(x)/**any_of**(x, …, vars)
matches(match)
:, e.g., mpg:cyl
!, e.g., !gear
everything()

MANIPULATE MULTIPLE VARIABLES AT ONCE

```
df <- tibble(x_1 = c(1, 2), x_2 = c(3, 4), y = c(4, 5))
```

across(.cols, .funs, …, .names = NULL) Summarize or mutate multiple columns in the same way.
df |> summarize(across(everything(), mean))

c_across(.cols) Compute across columns in row-wise data.

```
df |>
  rowwise() |>
  mutate(x_total = sum(c_across(1:2)))
```

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

vectorized function

mutate(.data, …, .keep = "all", .before = NULL, .after = NULL) Compute new column(s). Also **add_column()**.
mtcars |> mutate(gpm = 1 / mpg)
mtcars |> mutate(gpm = 1 / mpg, .keep = "none")

rename(.data, …) Rename columns. Use **rename_with()** to rename with a function.
mtcars |> rename(miles_per_gallon = mpg)

CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at **dplyr.tidyverse.org** • HTML cheatsheets at **pos.it/cheatsheets** • dplyr 1.1.4 • Updated: 2025-08

Operator Pipe %>% lub |>

```
# Bez pipe:
summarise(group_by(filter(dane, rok == 2023),
kraj), srednia = mean(wynik))

# Z pipe:
dane %>%
    filter(rok == 2023) %>%
    group_by(kraj) %>%
    summarise(srednia = mean(wynik))
```

Ctrl + Shift + M

WIZUALIZACJA

ggplot2 – Gramatyka grafiki

Tworzenie wykresów polega na dodawaniu kolejnych warstw:

Data

Jakie dane pokazujemy?

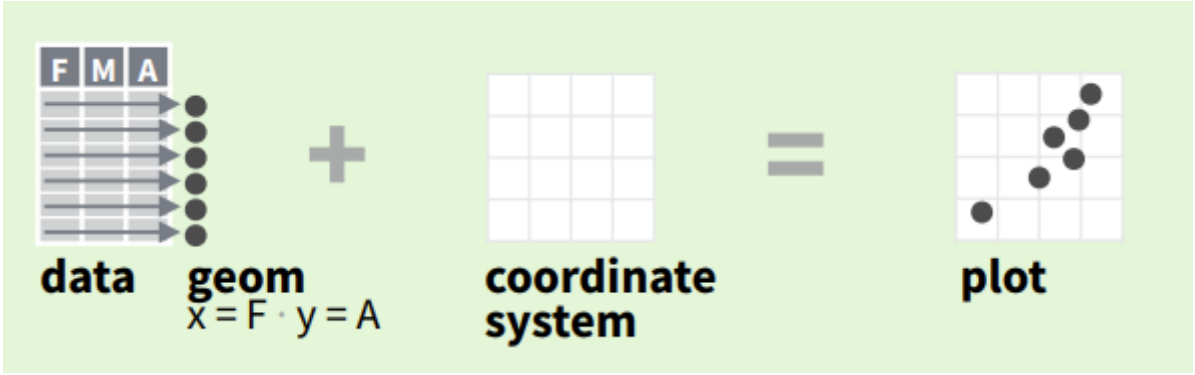
Aesthetics (aes)

Co jest na osi X, Y, jaki jest kolor/kształt?

Geom

Jaki to typ wykresu? (słupkowy, punktowy, liniowy).

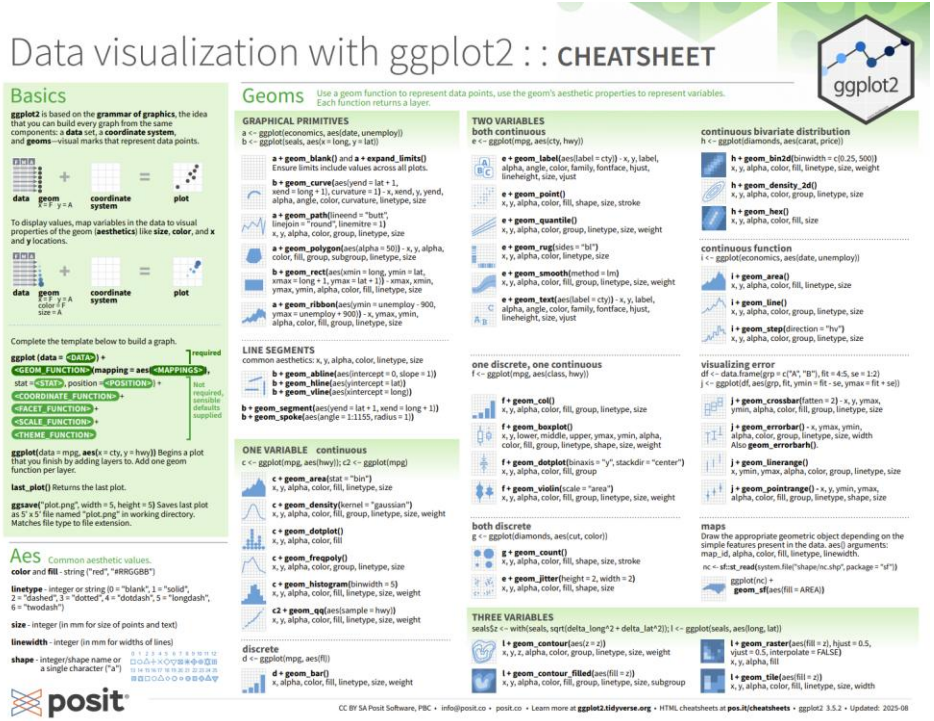
ggplot2 – Gramatyka grafiki



ggplot2 – Gramatyka grafiki

```
ggplot(dane, aes(x = wzrost, y = waga,
color = plec)) +
  geom_point() +
  theme_minimal()
```

Ggplot2 cheetsheet



Data visualization with ggplot2 : : CHEATSHEET

Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data** set, a **coordinate system**, and **geoms**—visual marks that represent data points.

data + geom (x = F, y = A) = coordinate system + plot

To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.

data + geom (x = F, y = A, color = F, size = A) = coordinate system + plot

Complete the template below to build a graph.

```
ggplot(data = <DATA>) +                                  required
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>),
  stat = <STAT>, position = <POSITION>) +                Not required, sensible defaults supplied
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION> +
  <SCALE_FUNCTION> +
  <THEME_FUNCTION>
```

ggplot(data = mpg, **aes**(x = cty, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

last_plot() Returns the last plot.

ggsave("plot.png", width = 5, height = 5) Saves last plot as 5' x 5' file named "plot.png" in working directory. Matches file type to file extension.

Aes

Common aesthetic values.

color and **fill** - string ("red", "#RRGGBB")

linetype - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dotdash", 5 = "longdash", 6 = "twodash")

size - integer (in mm for size of points and text)

linewidth - integer (in mm for widths of lines)

shape - integer/shape name or a single character ("a")

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

GRAPHICAL PRIMITIVES

```
a <- ggplot(economics, aes(date, unemploy))
b <- ggplot(seals, aes(x = long, y = lat))
```

a + geom_blank() and **a + expand_limits()** Ensure limits include values across all plots.

b + geom_curve(aes(yend = lat + 1, xend = long + 1), curvature = 1) - x, xend, y, yend, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend = "butt", linejoin = "round", linemitre = 1) x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size

b + geom_rect(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin = unemploy - 900, ymax = unemploy + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept = 0, slope = 1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend = lat + 1, xend = long + 1))
b + geom_spoke(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous

```
c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)
```

c + geom_area(stat = "bin") x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian") x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot() x, y, alpha, color, fill

c + geom_freqpoly() x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5) x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = hwy)) x, y, alpha, color, fill, linetype, size, weight

discrete

```
d <- ggplot(mpg, aes(fl))
```

d + geom_bar() x, alpha, color, fill, linetype, size, weight

TWO VARIABLES both continuous

```
e <- ggplot(mpg, aes(cty, hwy))
```

e + geom_label(aes(label = cty)) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_point() x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile() x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl") x, y, alpha, color, linetype, size

e + geom_smooth(method = lm) x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = cty)) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

one discrete, one continuous

```
f <- ggplot(mpg, aes(class, hwy))
```

f + geom_col() x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot() x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center") x, y, alpha, color, fill, group

f + geom_violin(scale = "area") x, y, alpha, color, fill, group, linetype, size, weight

both discrete

```
g <- ggplot(diamonds, aes(cut, color))
```

g + geom_count() x, y, alpha, color, fill, shape, size, stroke

e + geom_jitter(height = 2, width = 2) x, y, alpha, color, fill, shape, size

THREE VARIABLES

```
seals$z <- with(seals, sqrt(delta_long^2 + delta_lat^2)); l <- ggplot(seals, aes(long, lat))
```

l + geom_contour(aes(z = z)) x, y, z, alpha, color, group, linetype, size, weight

l + geom_contour_filled(aes(fill = z)) x, y, alpha, color, fill, group, linetype, size, subgroup

l + geom_raster(aes(fill = z), hjust = 0.5, vjust = 0.5, interpolate = FALSE) x, y, alpha, fill

l + geom_tile(aes(fill = z)) x, y, alpha, color, fill, linetype, size, width

continuous bivariate distribution

```
h <- ggplot(diamonds, aes(carat, price))
```

h + geom_bin2d(binwidth = c(0.25, 500)) x, y, alpha, color, fill, linetype, size, weight

h + geom_density_2d() x, y, alpha, color, group, linetype, size

h + geom_hex() x, y, alpha, color, fill, size

continuous function

```
i <- ggplot(economics, aes(date, unemploy))
```

i + geom_area() x, y, alpha, color, fill, linetype, size

i + geom_line() x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv") x, y, alpha, color, group, linetype, size

visualizing error

```
df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))
```

j + geom_crossbar(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar() - x, ymax, ymin, alpha, color, group, linetype, size, width
Also **geom_errorbarh()**

j + geom_linerange() x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange() - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps

Draw the appropriate geometric object depending on the simple features present in the data. aes() arguments: map_id, alpha, color, fill, linetype, linewidth.

```
nc <- sf::st_read(system.file("shape/nc.shp", package = "sf"))
ggplot(nc) +
  geom_sf(aes(fill = AREA))
```

posit

CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at **ggplot2.tidyverse.org** • HTML cheatsheets at **pos.it/cheatsheets** • ggplot2 3.5.2 • Updated: 2025-08

Dlaczego warto używać Tidyverse?

Kod jest **zrozumiały dla ludzi**, nie tylko dla komputerów.

Nauczysz się **jednego pakietu**, łatwiej zrozumiesz pozostałe.
Ogromne wsparcie online (Stack Overflow, blogi).

Współpracuje z modelowaniem danych, raportowaniem i danymi przestrzennymi.

PROGRAMOWANIE

Instrukcje warunkowe (if, else)

```
if (warunek) {
   # kod, gdy prawda
} else {
   # kod, gdy fałsz
}
```

Pętle

Pętla for: Wykonuje kod określoną liczbę razy (np. dla każdego elementu listy).

```
for (i in 1:5) {
  print(paste("Iteracja nr", i))
}
```

Pętle

Pętla while: Wykonuje kod dopóki warunek jest prawdziwy.

```
licznik <- 1

while (licznik <= 3) {
  print(paste("To jest krok numer:", licznik))
  licznik <- licznik + 1
}
```

Funkcje – Twoje własne narzędzia

!!!NIE POWTARZAJ SIĘ!!!

Jeśli kopiujesz i wklejasz ten sam fragment kodu **po raz trzeci** – czas na funkcję!

```
nazwa_funkcji <- function(argument1, argument2) {
  wynik <- argument1 + argument2
  return(wynik)
}
```

Funkcje – Twoje własne narzędzia

Zamiast 50 linii kodu, masz jedną nazwę, która mówi co się dzieje.

Zmieniasz kod w jednym miejscu, a poprawka działa wszędzie.

Mniejsze ryzyko pomyłki przy kopiowaniu zmiennych.